

Student Performance Prediction System

A Machine Learning based Academic Result Predictor with MySQL Integration

Project Report

Submitted by: Priyabrata Patra
Integrated M.Tech, Computer Science & Engineering (AI)
Vellore Institute of Technology, Bhopal
Course: AI and ML

2. Introduction

Academic performance prediction is a widely studied application of machine learning in the education domain. This project, the **Student Performance Prediction System**, is a complete end-to-end application that combines a relational database (MySQL), a data-cleaning pipeline, and two supervised machine learning classifiers to predict whether a student will **Pass** or **Fail** based on measurable study habits: daily study hours, attendance percentage, previous academic score, sleep hours, and participation in extracurricular activities.

Unlike a purely notebook-based ML exercise, this project is built as a working command-line application with full CRUD (Create, Read, Update, Delete) capability over a live MySQL database, so that the data an institution would actually hold on its students can be managed, cleaned, and used to train and query a predictive model, all from one interface.

3. Problem Statement

Educational institutions collect large amounts of data about their students (attendance, assignment scores, study patterns) but often lack a simple, low-cost tool to turn that data into an early-warning signal for students at risk of failing. Manually reviewing each student's record does not scale, and identifying the factors that most strongly predict failure is not obvious without a data-driven approach.

Goal: Build a system that (a) stores and manages student records in a structured database, (b) cleans and prepares that data for analysis, (c) trains a classification model capable of predicting Pass/Fail outcomes from a small set of behavioural and academic features, and (d) allows a user to query the trained model with a new student's details to get an instant prediction with confidence probabilities.

4. Functional Requirements

- The system shall allow a user to **view** all student records stored in the database.
- The system shall allow a user to **search** for a student by ID or by result (Pass/Fail).
- The system shall allow a user to **add** a new student record with study habit details.
- The system shall allow a user to **edit** an existing student record.
- The system shall allow a user to **delete** a student record, with confirmation.
- The system shall **clean** the dataset by correcting invalid values (e.g. negative study hours, out-of-range percentages) and filling missing values using the column median.
- The system shall **train** two classification models (Logistic Regression and Decision Tree) on the cleaned data and report their accuracy.
- The system shall allow a user to input a new, unseen student's details and receive a **predicted result** with class probabilities.
- The system shall display **feature importance** to explain which factors most influence the prediction.

5. Non-functional Requirements

- **Usability:** The system shall present a clear, numbered menu so a non-technical user can operate it without reading source code.
- **Reliability:** Database operations shall use parameterised queries to prevent SQL injection and shall confirm destructive actions (delete) before executing.
- **Performance:** Model training on a dataset of a few hundred rows shall complete in well under one second, keeping the CLI responsive.
- **Maintainability:** Each capability (CRUD, cleaning, training, prediction, evaluation) is isolated in its own function, so any one part can be modified without affecting the rest.
- **Data Integrity:** Global state (trained model, scaler, cleaned dataset) is only used after the relevant preparation step has run, with guard checks that prompt the user if a prerequisite step (cleaning, training) has not yet been completed.

6. System Architecture

The application follows a simple layered architecture: a **CLI presentation layer** that drives all user interaction, a **data access layer** that manages the MySQL connection and queries, a **data preparation layer** that cleans raw records using pandas, and a **machine learning layer** that trains and serves predictions using scikit-learn models. All persistent data lives in a single MySQL table (raw_students); the ML layer operates on an in-memory cleaned copy of this data for the duration of a session.

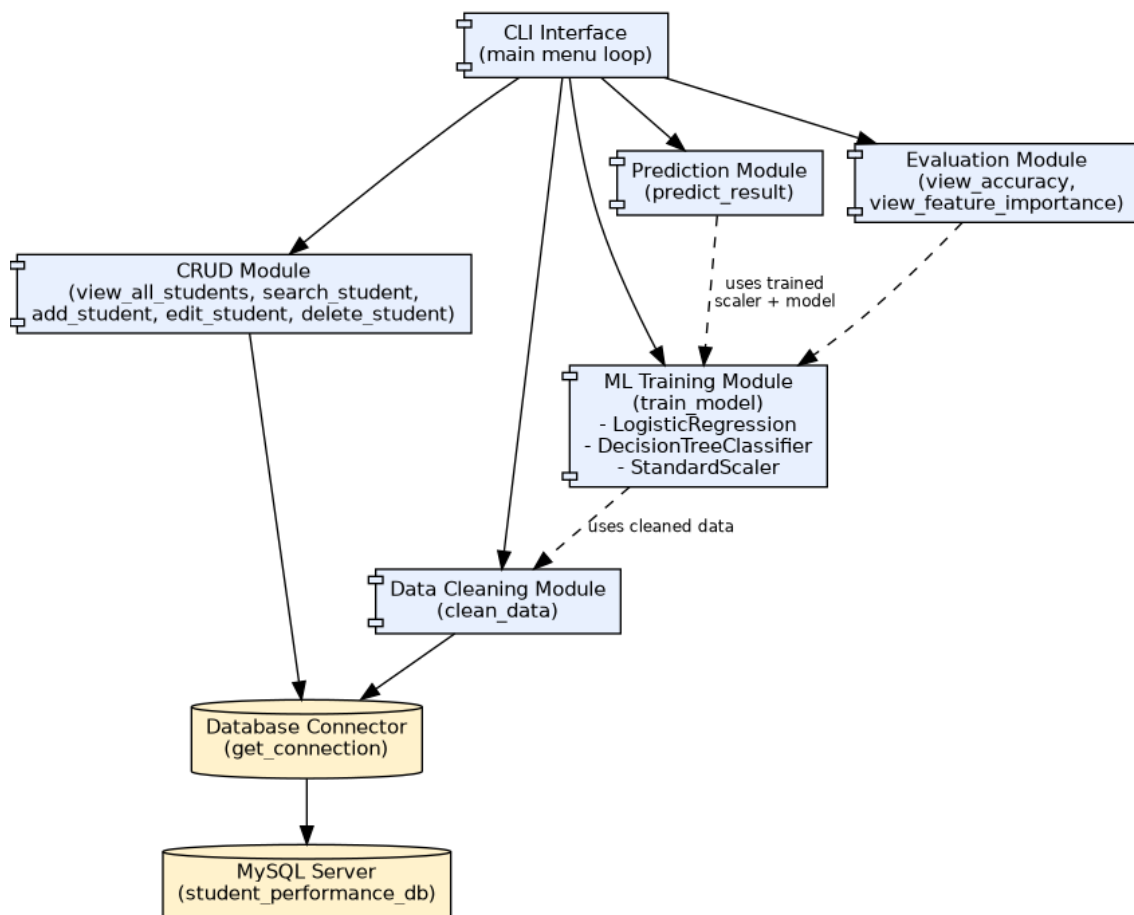


Figure 1: Component diagram showing the CLI, CRUD, cleaning, ML training, prediction and evaluation modules, and how they connect to MySQL.

7. Design Diagrams

7.1 Use Case Diagram

The single actor (**User**) can invoke ten distinct use cases exposed through the CLI menu, spanning record management and machine learning operations.

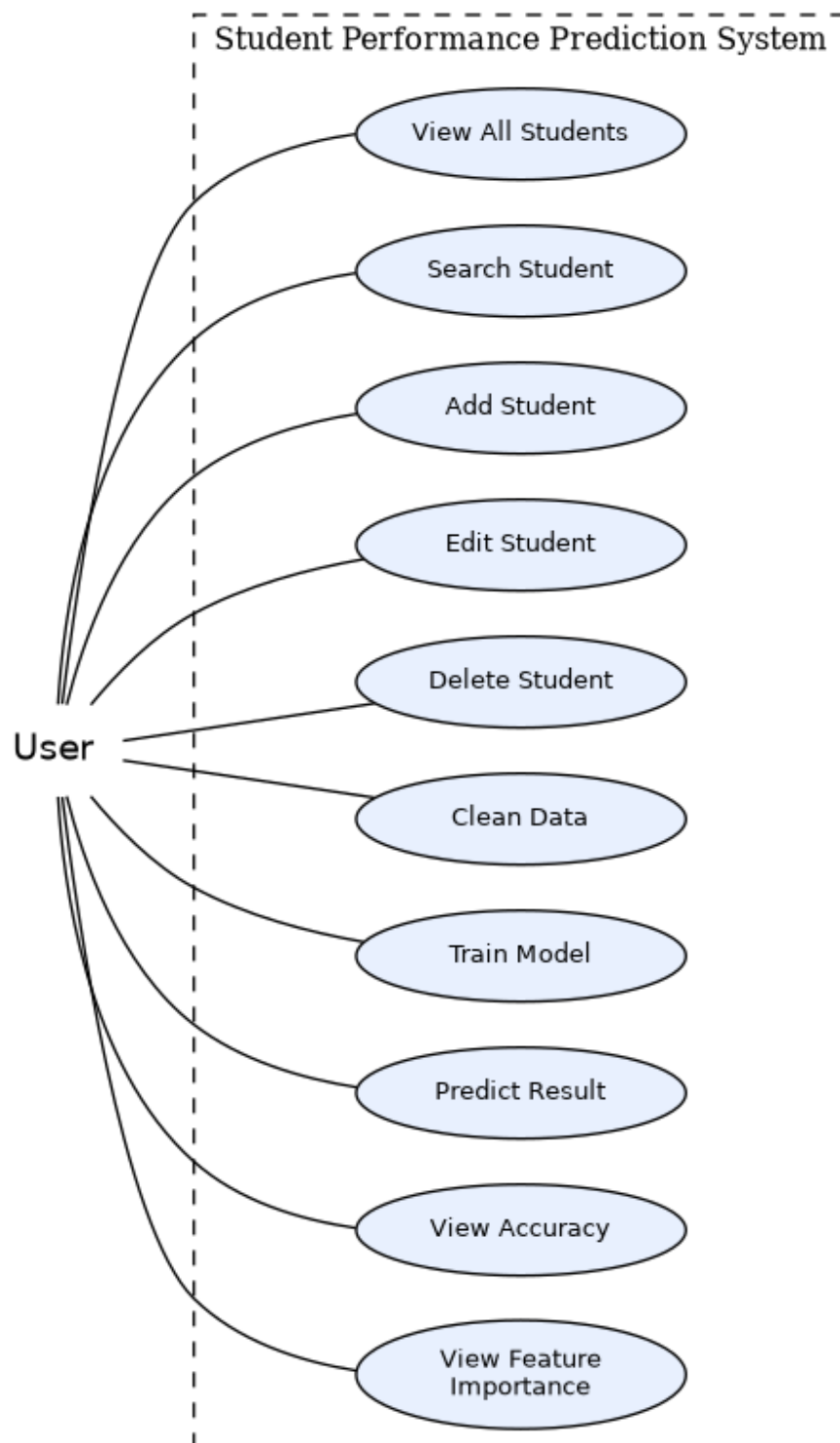


Figure 2: Use Case diagram of the Student Performance Prediction System.

7.2 Workflow Diagram

The workflow diagram traces the path from launching the CLI through to obtaining a prediction: CRUD operations feed the MySQL table, which is read and cleaned before the train/test split, model training, evaluation, and prediction steps.

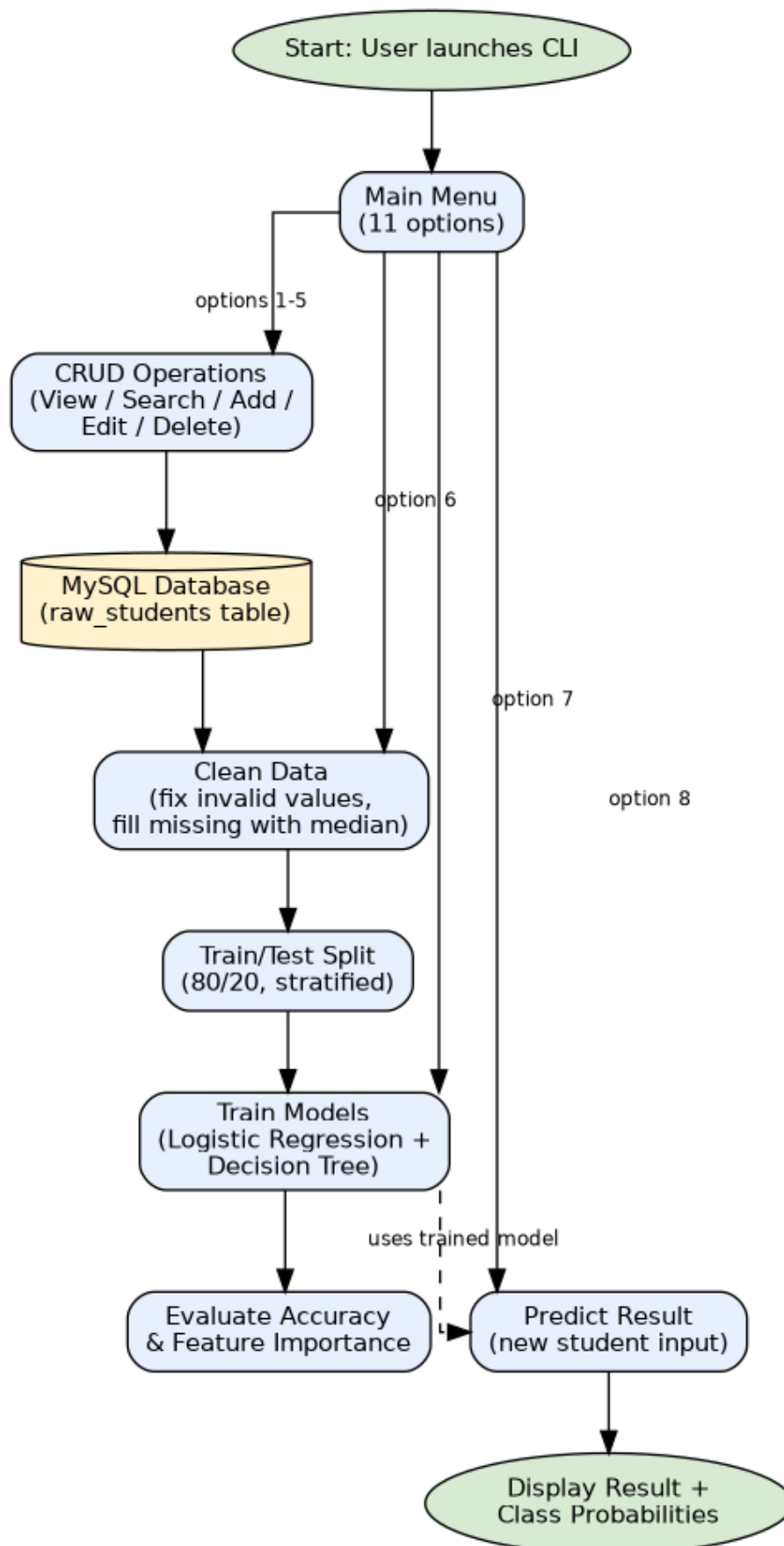


Figure 3: End-to-end workflow from CLI launch to displayed prediction.

7.3 Sequence Diagram

The sequence diagram below illustrates the full message flow for the **Predict Result** use case — the most involved interaction in the system, since it depends on data first being fetched, cleaned, and used to train a model before a new prediction can be served.

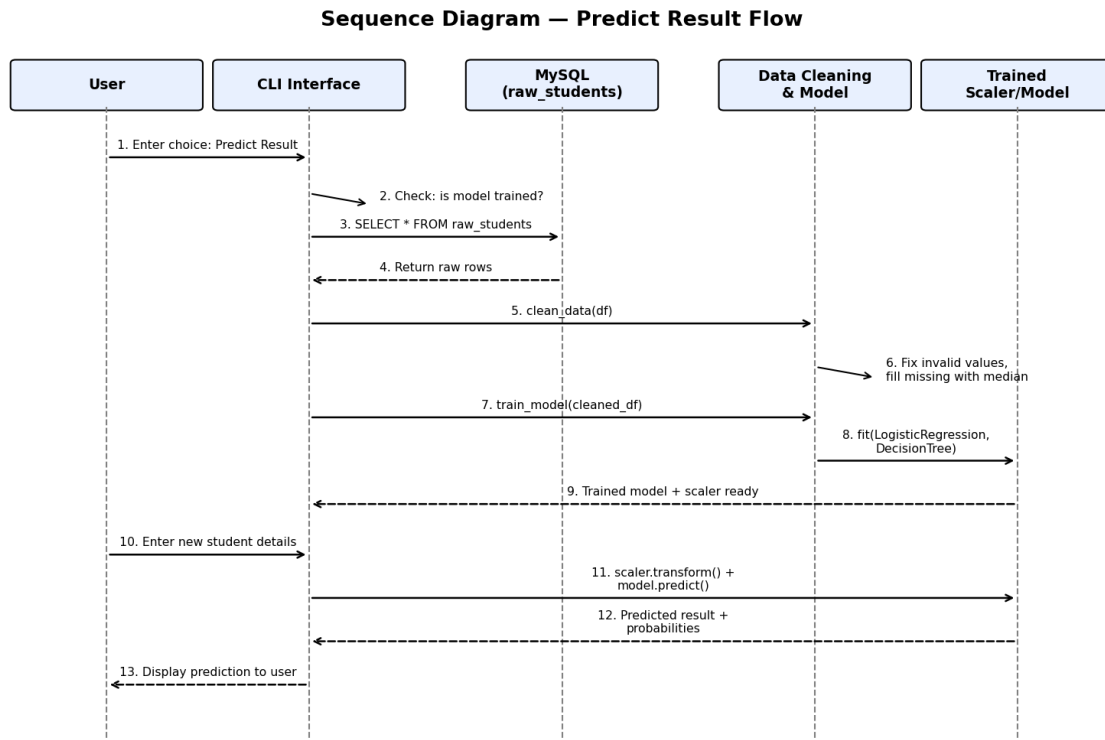


Figure 4: Sequence diagram for the Predict Result flow, from user input to displayed probabilities.

7.4 Class / Component Diagram

See Figure 1 in Section 6 (System Architecture) for the component diagram, which doubles as the class-level breakdown of responsibilities in this project, since the implementation is function-based rather than class-based.

7.5 ER Diagram

The system currently uses a single table, `raw_students`, which serves as both the live CRUD-managed dataset and the source data for model training. Two columns (`attendance_percent`, `previous_score`) are nullable by design, since real-world data collection can produce missing entries that the cleaning step is responsible for handling.

raw_students	
PK	id (INT, AUTO_INCREMENT)
	study_hours_per_day (FLOAT)
	attendance_percent (FLOAT, nullable)
	previous_score (FLOAT, nullable)
	sleep_hours (FLOAT)
	extracurricular (TINYINT)
	result (VARCHAR(10))

Figure 5: Entity-Relationship diagram of the raw_students table.

8. Design Decisions & Rationale

- **Single table design:** A single raw_students table was chosen over separate raw/cleaned tables to keep the schema simple for a CLI tool used interactively; cleaning is performed on an in-memory pandas copy rather than persisted separately.
- **Two models instead of one:** Logistic Regression and Decision Tree were both trained so that their accuracies could be directly compared, illustrating the trade-off between a simple linear model and a rule-based, non-linear model on the same data.
- **Median imputation over mean:** Missing numeric values are filled with the column median rather than the mean, since the median is more robust to the outliers and skew typically present in academic and attendance data.
- **Feature scaling for Logistic Regression only:** StandardScaler is applied before training Logistic Regression (which is sensitive to feature scale) but not before training the Decision Tree (which is scale-invariant), reflecting correct practice for each algorithm rather than applying one blanket preprocessing step.
- **Stratified train/test split:** The 80/20 split is stratified on the result column so that the proportion of Pass/Fail students is preserved in both the training and test sets, giving a fairer accuracy estimate on an imbalanced-leaning dataset.
- **Global state for the CLI session:** The trained model, scaler, and cleaned dataset are held in module-level variables rather than being re-computed on every menu action, so that a user can clean once, train once, and then predict multiple times without repeating expensive steps.

9. Implementation Details

The system is implemented as a single Python script, structured into clearly separated functions per responsibility. Key libraries used:

- **mysql-connector-python** — direct connection to the MySQL server for all CRUD and data-retrieval operations.

- **pandas** — loading query results into DataFrames, cleaning (type coercion, invalid-value removal, median imputation), and formatting console output.
- **NumPy** — numeric array handling for model input and marking invalid values as NaN prior to imputation.
- **scikit-learn** — `train_test_split`, `StandardScaler`, `LogisticRegression`, and `DecisionTreeClassifier` for the ML pipeline, along with `accuracy_score` and `classification_report` for evaluation.
- **matplotlib** — bar chart visualisation of Decision Tree feature importances.

Representative snippet — the cleaning step that fixes invalid values before imputation:

```
df.loc[df['study_hours_per_day'] < 0, 'study_hours_per_day'] = np.nan
df.loc[(df['attendance_percent'] < 0) | (df['attendance_percent'] > 100),
'attendance_percent'] = np.nan
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())
```

10. Screenshots / Results

10.1 Viewing Student Records

```
$ python3 student_performance_system.py [Option 1: View All Students]
```

===== ALL STUDENTS =====

id	study_hours_per_day	attendance_percent	previous_score	sleep_hours	extracurricular	result
1	6.0	70.1	76.4	6.9	0	Pass
2	4.7	73.3	NaN	6.0	1	Pass
3	6.3	89.0	78.0	6.5	1	Pass
4	8.0	87.3	85.3	8.0	1	Pass
5	4.5	79.7	71.2	6.7	1	Fail
6	4.5	81.4	93.2	6.6	1	Pass
7	8.2	95.3	53.4	4.9	0	Pass
8	6.5	72.9	46.3	7.4	0	Pass
9	4.1	86.6	38.3	7.3	1	Pass
10	6.1	77.6	87.4	9.1	0	Pass
11	4.1	77.4	74.8	6.1	1	Pass

Figure 6: Output of the 'View All Students' menu option, reading directly from MySQL.

10.2 Data Cleaning and Model Training

```
$ python3 student_performance_system.py [Options 6+7+9: Clean, Train, Accuracy]
```

```
===== DATA BEFORE CLEANING =====
```

```
Rows: 300
```

```
Missing values:
```

```
id          0
study_hours_per_day  0
attendance_percent  12
previous_score  8
sleep_hours  0
extracurricular  0
result       0
dtype: int64
```

```
===== DATA AFTER CLEANING =====
```

```
Missing values:
```

```
id          0
study_hours_per_day  0
attendance_percent  0
previous_score  0
sleep_hours  0
extracurricular  0
result       0
dtype: int64
```

```
Data cleaning completed.
```

```
===== MODEL TRAINING =====
```

```
Logistic Regression Accuracy: 65.00%
```

```
Decision Tree Accuracy: 61.67%
```

```
Best Model: Logistic Regression
```

```
===== MODEL ACCURACY =====
```

```
Logistic Regression: 65.00%
```

```
Decision Tree: 61.67%
```

```
Best Model: Logistic Regression
```

Figure 7: Console output for data cleaning (12 and 8 missing values found and fixed) followed by model training and accuracy comparison.

10.3 Prediction Output

```
$ python3 student_performance_system.py [Option 8: Predict Result]
```

```
===== PREDICTION =====
```

```
Predicted Result: Pass
```

```
Fail probability: 12.95%
```

```
Pass probability: 87.05%
```

Figure 8: Prediction for a new, unseen student (6.5 study hrs/day, 92% attendance) resulting in a Pass prediction with 87.05% probability.

10.4 Feature Importance

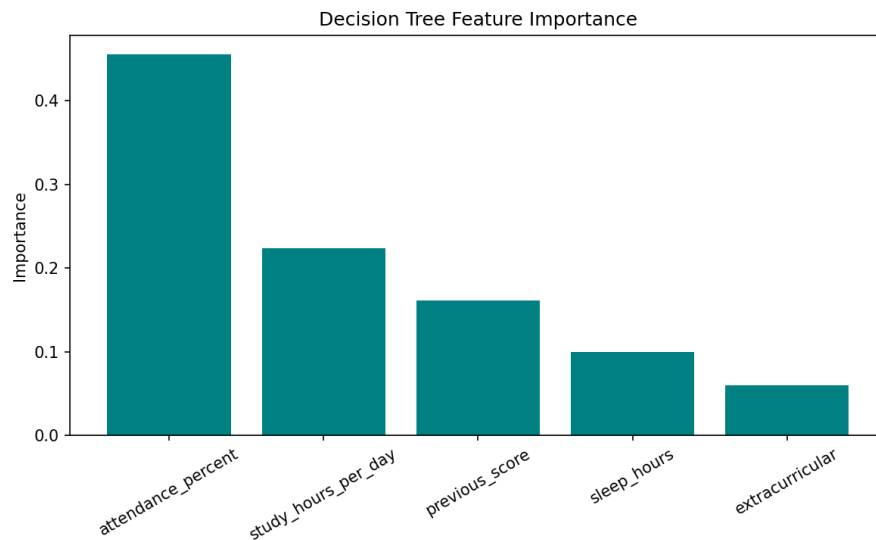


Figure 9: Decision Tree feature importances, showing which factors the model relied on most heavily when making a prediction.

11. Testing Approach

Each menu function was tested individually against a live MySQL database seeded with 300 synthetic student records (containing intentionally injected missing values and invalid entries to validate the cleaning logic). Testing was performed by simulating user input programmatically and verifying the resulting state directly in the database with SQL queries, rather than relying on visual inspection alone.

Function	Test performed	Result
view_all_students	Query all rows, check row count matches DB	Pass
search_student	Search by ID and by result, verify row(s) returned	Pass
add_student	Insert new record, verify via direct SQL SELECT	Pass
edit_student	Update a record, verify new values via SQL	Pass
delete_student	Delete a record, verify row count decreased	Pass
clean_data	Verify missing-value count is 0 after cleaning	Pass
train_model	Verify both models train and report accuracy	Pass
predict_result	Predict on new input, verify probabilities sum to 100%	Pass
view_feature_importance	Verify chart generation after training	Pass

Edge cases such as an invalid menu choice, a non-numeric input for numeric fields, and attempting to predict before training was completed were also verified to fail gracefully with a clear message rather than crashing the program.

12. Challenges Faced

- **Messy real-world-style data:** Deliberately introducing missing values and invalid entries (negative study hours, out-of-range percentages) meant the cleaning logic had to be genuinely robust rather than assuming well-formed input.
- **Scaling only where needed:** Understanding that Logistic Regression requires scaled features while Decision Trees do not, and structuring the code so each model receives the correct version of the data, required care to avoid accidentally degrading one model's performance.
- **Keeping the CLI stateful but safe:** Using module-level globals for the trained model and scaler simplified the menu-driven flow, but required guard checks (e.g. "please train the model first") to prevent the program from crashing if a user jumped to prediction before training.
- **Balancing realism with learnability:** The synthetic dataset was tuned so that model accuracy would land in a realistic 60–70% range rather than an unrealistic 100%, which better reflects genuine classification difficulty and avoids masking overfitting.

13. Learnings & Key Takeaways

- Gained practical experience connecting a Python application to a live MySQL database for both transactional (CRUD) and analytical (ML) workloads using the same underlying table.
- Learned why different ML algorithms require different preprocessing (e.g. feature scaling for Logistic Regression but not Decision Trees) rather than applying one pipeline to all models uniformly.
- Understood the practical importance of data cleaning — specifically distinguishing between missing values (imputed) and invalid/impossible values (corrected before imputation) — as a prerequisite to reliable model training.
- Practised structuring a full-stack-style ML project (database + cleaning + training + serving predictions) as a single cohesive tool rather than disconnected notebook cells.

14. Future Enhancements

- Persist predictions back into a dedicated students results table (actual vs. predicted) to enable historical tracking and further analysis, rather than only showing predictions at the CLI in real time.
- Add a web-based front end (e.g. Flask or Streamlit) so the system can be used without a terminal, widening accessibility for non-technical staff.
- Introduce additional models (Random Forest, k-Nearest Neighbours) for a broader comparison, and add cross-validation instead of a single train/test split for more robust accuracy estimates.
- Expand the feature set with real institutional data (assignment scores, quiz attempts) once available, replacing the synthetic dataset used for this project.
- Add authentication so that only authorised staff can add, edit, or delete student records.

15. References

- scikit-learn documentation — <https://scikit-learn.org/stable/documentation.html>
- pandas documentation — <https://pandas.pydata.org/docs/>
- MySQL Connector/Python Developer Guide — <https://dev.mysql.com/doc/connector-python/en/>
- matplotlib documentation — <https://matplotlib.org/stable/index.html>
- Course materials and lecture content from the AI and ML course, VIT Bhopal.